1) FEC

```
module fec (input wire clock,
            input wire start,
            input wire data,
            output logic done,
            output logic [95:0] fec
            );
        parameter MAX = 48;
        logic [2:0] x;
        logic [6:0] cycle_counter;
        always_ff @(posedge clock) begin
            if(start) begin
            //Resests State Variables
                x <= 0;
                done <= 0;
                fec <= 0;
                cycle_counter <= 0;
            end else begin
               if (cycle_counter ==  MAX - 1) begin
                    done <= 1;
                    cycle_counter <= 0;
                    //Asserts done after 48 cycles
               end else begin
                    cycle_counter <= cycle_counter + 1;
                    done <= 0;
               end
               //We Calculate the FEC concurrently and keep three registers to keep th
e state of the window
               fec <= {fec, {data ^ x[1] ^ x[2]},{data ^ x[0] ^ x[1] ^ x[2]}};
               x[0] <= data;
               x[1] <= x[0];
               x[2] <= x[1];
            end
        end
endmodule
```

1.1) FEC TESTBENCH

```
`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////////////////////
//
// lpset FEC test bench  9/25/2019
//
////////////////////////////////////////////////////////////////////////////////

module fec_tb;

   // Inputs
   logic clock;
   logic data_clk;
   logic start;
   logic data;

   // Outputs
   logic done;
   logic [95:0] fec;



   // Instantiate the Unit Under Test (UUT)
   fec uut (
      .clock(clock),
      .start(start),
      .data(data),
```

```
      .done(done),
      .fec(fec)
   );

 // this is the input data
   logic [47:0] input_data = 48'h03_01_02_03_30_3A;
   integer i;       // required for "for" loop

   initial begin    // system clock
      forever #5 clock = !clock;
      end

   initial begin    // data_clk, ensures setup time met
      #2
      forever #5 data_clk = !data_clk;
      end

   initial begin
      // Initialize Inputs
      clock = 0;
      data_clk = 0;
      start = 0;
      data = 0;

      // Wait 100 ns for global reset to finish
      #100;

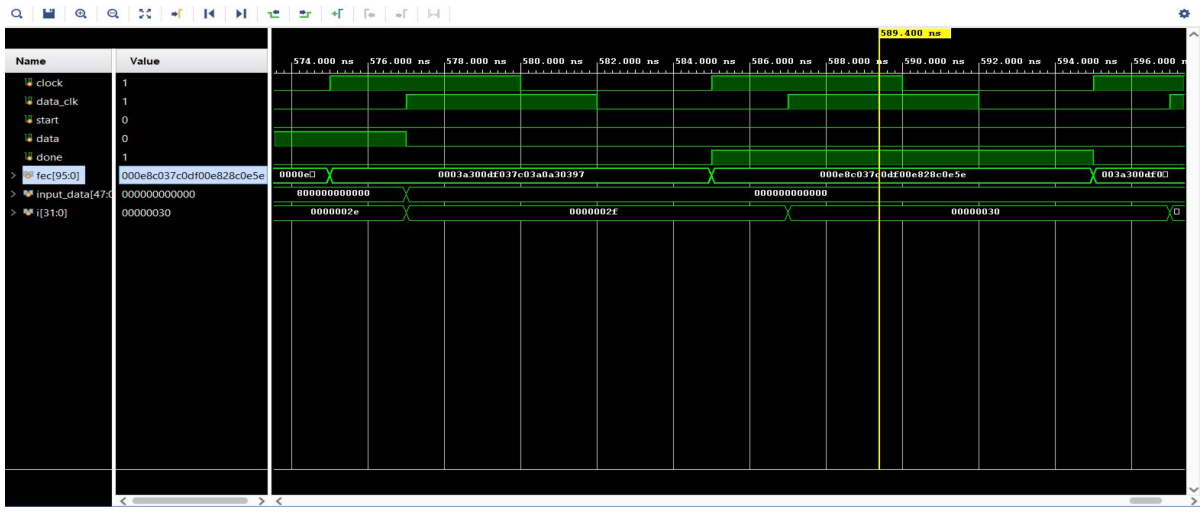
      // Add stimulus here
      start=1;
      #10 start = 0;
      #5;        // change if necessary for your design

      //forever #5 data_clk = !data_clk;
       for (i=0; i<50; i=i+1)
       begin
         data = input_data[47];
         // at each clock, left shift the data
         // note syntax for test bench "for" loop - no "always"
         // note the blocking assignment (immediate)
         @(posedge data_clk) input_data = {input_data[46:0],1'b0};
       end

   $stop; // Pause simulation
   end

endmodule
```

1.2) FEC SCREENSHOT



2) Full System

```
//////////////////////////////////////////////////////////////////////////////////
// Company: MIT
// Engineer: Ahmad Taka
//
// Create Date: 10/20/2021 09:40:57 PM
// Design Name:
// Module Name: top_level
// Project Name: Communication Lpset 11
// Additional Comments:
//      This is my full top level for the FEC Communication system
//////////////////////////////////////////////////////////////////////////////////

`default_nettype none

module top_level(
    input wire clock,
    input wire start,
    input wire data,
    output logic done,
    output logic [6:0] cycle_counter,
    output logic [95:0] fec_out,
    output logic control
    );
    //This Module only uses only two state variables one for montioring the state of t
he crc calculation
    // and another to store the result of that calculation. Note a third to store the
output of crc concurrently.

    logic [15:0] r_temp;


    logic [15:0] r;
    crc crc1 (.clock(clock), .start(start), .data(data),
        .done(control), .r(r));

    logic fec_data;
    fec fec1 (.clock(clock), .start(start), .data(fec_data),
        .done(done), .fec(fec_out));

    always_comb begin
        if(control) begin
            fec_data = r_temp[15];
            r_temp = r;
        end else begin
            fec_data = data;
        end
    end
    always_ff @(posedge clock)  begin
        if (start) begin
            cycle_counter <= 0; //
            r_temp <= 0; // sets states to 0
        end else begin
            if (cycle_counter == 47) begin
                cycle_counter <= 0; //After 48 Cycles reset count
            end else begin
```

```
                cycle_counter <= cycle_counter + 1;
            end
        end
        if (control) begin
            r_temp <= r_temp << 1;
        end
    end
endmodule

module crc(
    input wire clock,
    input wire start,
    input wire data,
    output logic done,
    output logic [15:0] r
    );
    parameter COUNT = 32;
    logic [5:0] bit_counter; //Counts to 32 bits
    always_ff @(posedge clock) begin
        if (start) begin
            //Initial Trigger
            r <= 16'hFFFF;
            done <= 1'b0;
            bit_counter <= 1'd0;
        end else begin
            if (bit_counter == COUNT - 1) begin
            //Termination
                done <= 1'b1;
                bit_counter <= 1'd0;
            end else begin
                done <= 1'b0;
                bit_counter <= bit_counter + 1'd1;
             end
             r[0] <= data + r[15];
             r[1] <= r[0];
             r[2] <= r[15] + r[1] + data;
             for(int i = 3; i < 15; i++) begin r[i] <= r[i - 1]; end //All registers 3
-14 are equal to previous
            r[15] <= r[14] + r[15] + data;
        end
    end
endmodule

module fec (input wire clock,
            input wire start,
            input wire data,
            output logic done,
            output logic [95:0] fec
            );
        parameter MAX = 48;
        logic [2:0] x;
        logic [6:0] cycle_counter;
        always_ff @(posedge clock) begin
            if(start) begin
            //Resests State Variables
                x <= 0;
                done <= 0;
                fec <= 0;
                cycle_counter <= 0;
            end else begin
               if (cycle_counter ==  MAX - 1) begin
                    done <= 1;
                    cycle_counter <= 0;
                    //Asserts done after 48 cycles
               end else begin
```

```
                    cycle_counter <= cycle_counter + 1;
                    done <= 0;
                end
                //We Calculate the FEC concurrently and keep three registers to keep th
e state of the window
                fec <= {fec, {data ^ x[1] ^ x[2]},{data ^ x[0] ^ x[1] ^ x[2]}};
                x[0] <= data;
                x[1] <= x[0];
                x[2] <= x[1];
            end
        end
endmodule

`default_nettype wire
```

2.1) Full System Testbench

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
//
// lpset Top_Level test bench  9/25/2019
//
//////////////////////////////////////////////////////////////////////////////////

module top_level_tb;

   // Inputs
   logic clock;
   logic data_clk;
   logic start;
   logic data;

   // Outputs
   logic done;
   logic [95:0] fec;
   logic [6:0] cycle_counter;
   logic control;

   // Instantiate the Unit Under Test (UUT)
   top_level uut (
      .clock(clock),
      .start(start),
      .data(data),
      .done(done),
      .fec_out(fec),
      .cycle_counter(cycle_counter),
      .control(control)
   );

 // this is the input data
   logic [47:0] input_data = 48'h03_01_02_03_30_3A;
   integer i;      // required for "for" loop

   initial begin   // system clock
      forever #5 clock = !clock;
      end

   initial begin   // data_clk, ensures setup time met
      #2
      forever #5 data_clk = !data_clk;
      end

   initial begin
      // Initialize Inputs
```

```
        clock = 0;
        data_clk = 0;
        start = 0;
        data = 0;

        // Wait 100 ns for global reset to finish
        #100;

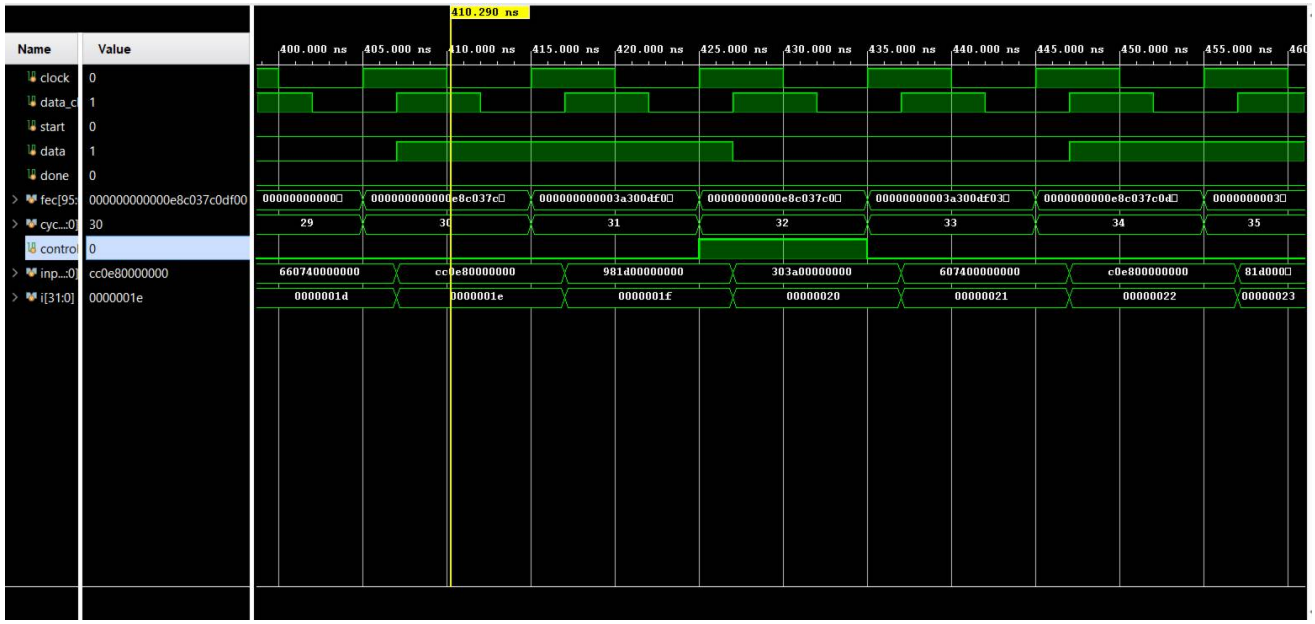
        // Add stimulus here
        start=1;
        #10 start = 0;
        #5;        // change if necessary for your design

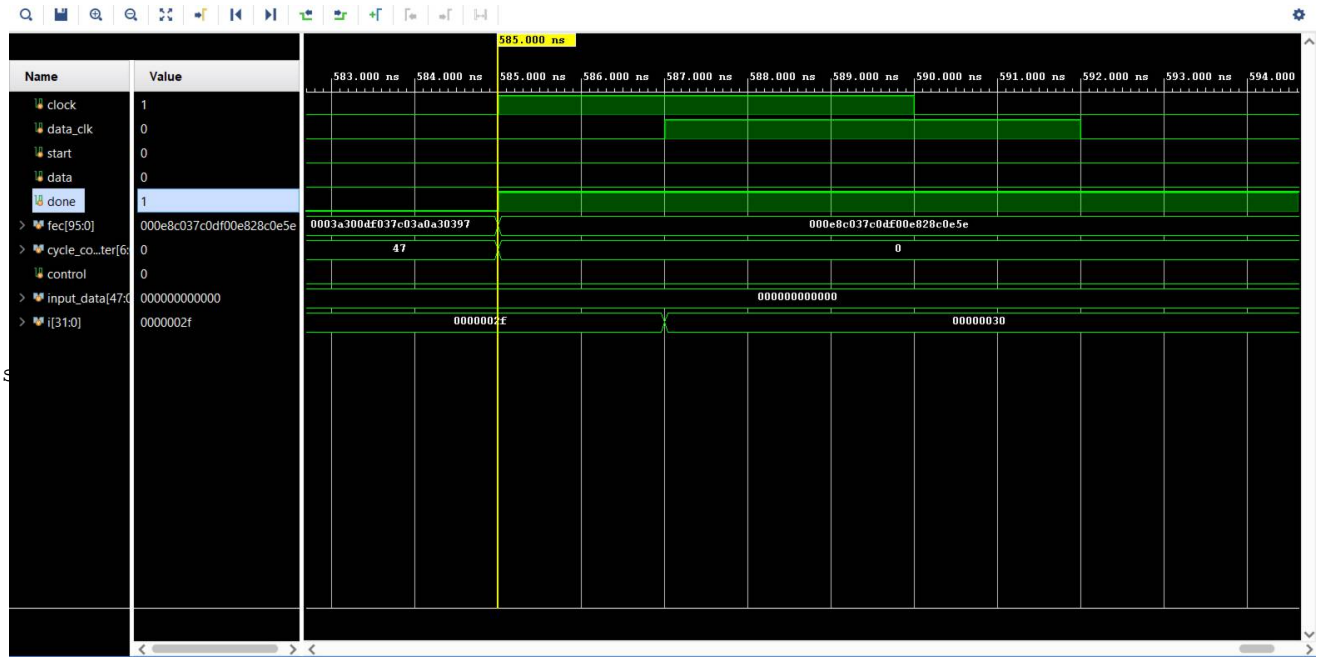
        //forever #5 data_clk = !data_clk;
         for (i=0; i<50; i=i+1)
         begin
           data = input_data[47];
           // at each clock, left shift the data
           // note syntax for test bench "for" loop - no "always"
           // note the blocking assignment (immediate)
           @(posedge data_clk) input_data = {input_data[46:0],1'b0};
         end

    $stop; // Pause simulation
    end

endmodule
```

2.2) Screenshots




585.000 ns
583.000 ns
584.000 ns
585.000 ns
586.000 ns
587.000 ns
588.000 ns
589.000 ns
590.000 ns
591.000 ns
592.000 ns
593.000 ns
594.000
Name
Value
clock
1
data_clk
0
start
0
data
0
done
1
fec[95:0]
000e8c037c0df00e828c0e5e
0003a300df037c03a0a30397
000e8c037c0df00e828c0e5e
cycle_co...ter[6:
0
47
0
control
0
input_data[47:0
000000000000
000000000000
i[31:0]
0000002f
0000002f
00000030